# WiCyS CTF 2024
# Johnny Torrance Challenge

## AUTHOR

Chloe Quinn
Discord: @realpsyop
Email: chq8025@rit.edu

## ABOUT

This years theme is Escape Artists.
Characters from your favorite horror stories have escaped!
As a challenge author you should choose a character from your favorite horror novel (get obscure!) and create a challenge that will help the players find / "catch" the character.
What has the character done after getting out of the book, where did they go, what chaos might they be making?

*October 20, 2024*

# Challenge Overview

**Challenge Name:** Here's Johnny!
**Category:** Cryptography
**Difficulty:** Intermediate to Hard
**Author:** Chloe Q. (@realpsyop)

John "Jack" Torrance stumbled out of The Shining into a fog-drenched town, clutching a cyber-weapon that pulsed with dark energy. Driven by madness, he scrawled two chilling **messages** on a wall:

S8NNIMA3XylLsUwjRjSLJXk5QHhGzVu−dBZdwL5−RzxK2r930KO7wsI−eXdaPmtq_8I9TQPZfTOMYuZoU9PW01R9g0pFxIfUbnRJTkc=

**Please note that when copying from the PDF, the hyphens will not copy properly. I will send the encrypted message in Discord along with attaching this file to ensure no confusion.**

## Included Public Hint Image

"Blood" on the Wall Picture

Decipher his cryptic warnings to protect the townsfolk and apprehend Johnny before it is too late! Unravel the clue he has left behind to uncover the truth and ensure the safety of everyone in the community.

## Private Information

**Easiest to solve using:** Python or Unix
**Tools Used:** Base64, XOR, gzip
**Flag:** WCTF{D0CT0R_SL33P_H3RES_J0HNNY_G03SCrazYFR_SuCh_a_C14551C}

# Base64

When viewing the encrypted message, it can be very easily inferred that it is Base64. Specifically URLSafe encoded Base64.

## **Encrypted Message**:

```
S8NNIMA3XylLsUwjRjSLJXk5QHhGzVu-dBZdwL5-RzxK2r930KO7wsI-eXdaPmtq_8I9
    TQPZfTOMYuZoU9PW01R9g0pFxIfUbnRJTkc=
```

To decode URLSafe Base64 in python, we can use:

```
import base64

inf = input("Input: ")
encoded_data = inf.encode()

decoded_data = base64.urlsafe_b64decode(encoded_data)
print(decoded_data)
```

When URLSafe Base64 decoding, we get the following data:

```
b'K\xc3M \xc07_)K\xb1L#F4\x8b%y9@xF\xcd[\xbet\x16]\xc0\xbe~G<J\xda\xbfw\xd0\xa3
    \xbb\xc2\xc2>ywZ>kj\xff\xc2=M\x03\xd9}3\x8cb\xe6hS\xd3\xd6\xd3T}\x83JE\xc4\
    x87\xd4ntING'
```

This information can then be interpreted as a XOR encrypted message.

# XOR

The XOR part of the challenge has the key 'THE SHINING' based on the image, this isn't too 'guessy', you just need to pay attention to the image.

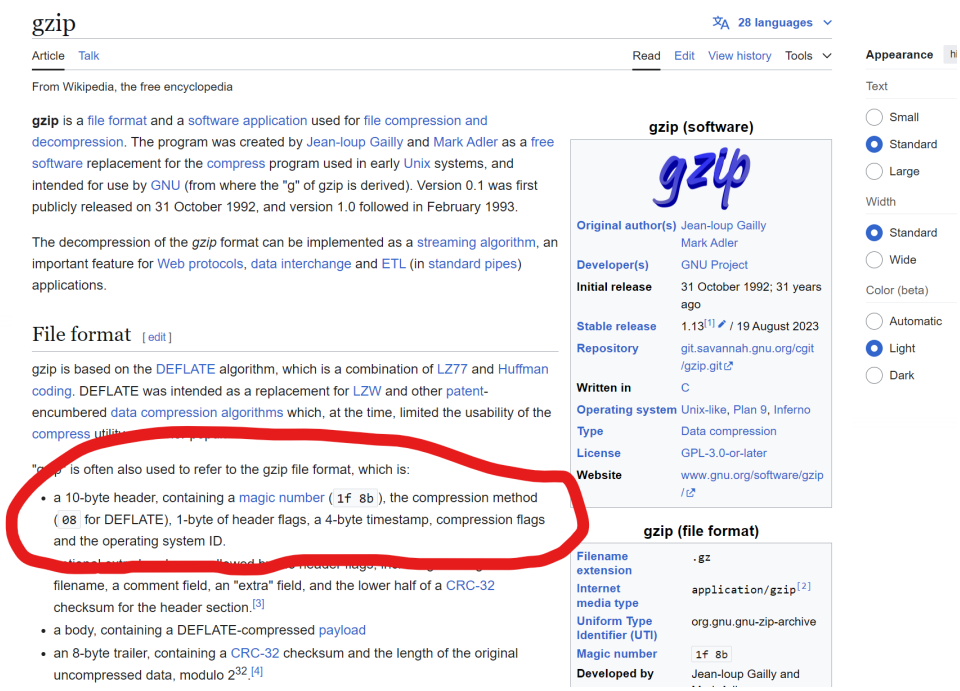
To XOR text using a key of 'THE SHINING' in Python, we can use:

```
key = 'THE SHINING'
xored = bytearray([b ^ ord(key[i % len(key)]) for i, b in enumerate(decoded_data)])
print(xored)
```

The result is:

```
bytearray(b"\x1f\x8b\x08\x00\x93\x7f\x16g\x02\xff\x0bw\x0eq\xabv1p\x0e1\x08\x8a
    \x0f\xf616\x0e\x88\xf70\x0er\r\x8e\xf72\xf0\xf0\xf3\x8b\x8cw70\x0ev.J\xac\
    x8at\x03J\x97:g\xc4\'\xc6;\x1b\x9a\x98\x9a\x1a:\xd7\x02\x00\xe4\xd4\x9c\':\
    x00\x00\x00")
```

Based on the official gzip wiki page and many other sources, the magic hexadecimal numbers are at the start (0x1F, 0x8B, 0x08), which indicates a gzip header! Wikipedia Source



gzip

28 languages

Article Talk Read Edit View history Tools

From Wikipedia, the free encyclopedia

**gzip** is a file format and a software application used for file compression and decompression. The program was created by Jean-loup Gailly and Mark Adler as a free software replacement for the compress program used in early Unix systems, and intended for use by GNU (from where the "g" of gzip is derived). Version 0.1 was first publicly released on 31 October 1992, and version 1.0 followed in February 1993.

The decompression of the *gzip* format can be implemented as a streaming algorithm, an important feature for Web protocols, data interchange and ETL (in standard pipes) applications.

File format [ edit ]

gzip is based on the DEFLATE algorithm, which is a combination of LZ77 and Huffman coding. DEFLATE was intended as a replacement for LZW and other patent-encumbered data compression algorithms which, at the time, limited the usability of the compress utility.

"gzip" is often also used to refer to the gzip file format, which is:

- a 10-byte header, containing a magic number (`1f 8b`), the compression method (`08` for DEFLATE), 1-byte of header flags, a 4-byte timestamp, compression flags and the operating system ID.
- filename, a comment field, an "extra" field, and the lower half of a CRC-32 checksum for the header section.[3]
- a body, containing a DEFLATE-compressed payload
- an 8-byte trailer, containing a CRC-32 checksum and the length of the original uncompressed data, modulo $2^{32}$.[4]

gzip (software)

| | |
|---|---|
| Original author(s) | Jean-loup Gailly, Mark Adler |
| Developer(s) | GNU Project |
| Initial release | 31 October 1992; 31 years ago |
| Stable release | 1.13[1] / 19 August 2023 |
| Repository | git.savannah.gnu.org/cgit/gzip.git |
| Written in | C |
| Operating system | Unix-like, Plan 9, Inferno |
| Type | Data compression |
| License | GPL-3.0-or-later |
| Website | www.gnu.org/software/gzip/ |

gzip (file format)

| | |
|---|---|
| Filename extension | .gz |
| Internet media type | application/gzip[2] |
| Uniform Type Identifier (UTI) | org.gnu.gnu-zip-archive |
| Magic number | `1f 8b` |
| Developed by | Jean-loup Gailly and |

Appearance

Text: Small, Standard, Large

Width: Standard, Wide

Color (beta): Automatic, Light, Dark

# gzip

This part of the challenge is honestly the easiest part, it is either one line in Python, Unix or something else. I'm quite sure you can even use some sort of zipping software if you really wanted to.

**Text that is encrypted** with gzip:

```
bytearray(b"\x1f\x8b\x08\x00\x93\x7f\x16g\x02\xff\x0bw\x0eq\xabv1p\x0e1\x08\x8a
   \x0f\xf616\x0e\x88\xf70\x0er\r\x8e\xf72\xf0\xf0\xf3\x8b\x8cw70\x0ev.J\xac\
   x8at\x03J\x97:g\xc4\'\xc6;\x1b\x9a\x98\x9a\x1a:\xd7\x02\x00\xe4\xd4\x9c\':\
   x00\x00\x00")
```

To decompress using gzip in Python is INSANELY simple:

```
decompressed_data = gzip.decompress(xored)
print("Output:",decompressed_data.decode())
```

Here we finally get the flag!

```
Output: WCTF{D0CT0R_SL33P_H3RES_J0HNNY_G03SCrazYFR_SuCh_a_C14551C}
```

Full Proof of Concept decryption and encryption programs written in Python are on the next page.

# Proof of Concept Encryption/Decryption

## Decryption in Python:

```
import base64
import gzip
inf = input("Input: ")
encoded_data = inf.encode()

decoded_data = base64.urlsafe_b64decode(encoded_data)
print(decoded_data)

key = 'THE SHINING'
xored = bytearray([b ^ ord(key[i % len(key)]) for i, b in enumerate(
    decoded_data)])
print(xored)
decompressed_data = gzip.decompress(xored)

print("Output:",decompressed_data.decode())
```

## Encryption in Python:

```
from itertools import cycle
import base64
import gzip

inf = input("Input: ")
data = bytes(inf, 'utf-8')

compressed_data = gzip.compress(data)

key = 'THE SHINING'
xored = "".join(chr(ord(x) ^ ord(y)) for (x, y) in zip(compressed_data.decode('
    latin1'), cycle(key)))

encoded_data = base64.b64encode(xored.encode('latin1'), b"-_")

print("Output:",str(encoded_data.decode('utf-8')))
```